Lab 9

JDBC with PostgreSQL

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Lab Workflow **2**

2 Database Schema & Data File **4**

3 Environment Setup **6**
3.1 System Installation 6
3.2 VS Code Configuration 6

4 Understanding JDBC **7**
4.1 What is JDBC? 7
4.2 How It Works: The Architecture 7
4.3 Core Components & Syntax 7
4.4 Prepared Statements 7
4.5 Real-Life Application 8

5 JDBC Code Examples **9**
5.1 Basic Select (Using Statement) 9
5.2 How to Run the Code 9

6 Scenario **10**

7 Lab Task **10**

8 Useful Syntax & Coding Guidelines **11**
8.1 Retrieving Values from ResultSet 11
8.2 Filtering by Date 11
8.3 Calculating Balance & Volume 11
8.4 PreparedStatement Syntax 12

9 PostgreSQL in Python **13**
9.1 1. Installation 13
9.2 2. Python Code Example 13

1 Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

   1. Connect to Your Database:

   Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Verify the Connection:

   Verify that you are connected to the correct database with the following command:

```
\conninfo
```

   Expected output:

```
Database: cse4308_12345, User: student_12345
```

   3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

   Load the SQL file that sets up the tables and inserts initial data.

   1. Load the SQL File:

   Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_09_setup.sql
```

   This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

   After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

   1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Check Tables:

   Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L9_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

2 Database Schema & Data File

Instructions: Copy the code below and save it as a file named `lab_09_setup.sql` in your `Downloads` directory. This file will be used in Step 6 of the Lab Workflow.

```
-- 1. CLEANUP
DROP TABLE IF EXISTS TRANSACTIONS;
DROP TABLE IF EXISTS ACCOUNT;

-- 2. CREATE TABLES
CREATE TABLE ACCOUNT (
    A_ID INT PRIMARY KEY
);

CREATE TABLE TRANSACTIONS (
    T_ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DTM TIMESTAMP,
    A_ID INT,
    AMOUNT DECIMAL(15,2),
    TYPE INT, -- 0 for Credit, 1 for Debit
    FOREIGN KEY (A_ID) REFERENCES ACCOUNT(A_ID)
);

-- 3. INSERT ACCOUNTS (Generates 100 Accounts)
INSERT INTO ACCOUNT (A_ID)
SELECT generate_series(1, 100);

-- 4. INSERT TRANSACTIONS (Generates ~1000 random transactions)
INSERT INTO TRANSACTIONS (DTM, A_ID, AMOUNT, TYPE)
SELECT
    timestamp '2020-01-01' +
    random() * (timestamp '2022-12-31' - timestamp '2020-01-01'),
    floor(random() * 100 + 1)::int,
    (random() * 990000 + 10000)::decimal(15,2),
    floor(random() * 2)::int
```

```
FROM generate_series(1, 1000);

COMMIT;
```

Listing 1: lab_09_setup.sql

3 Environment Setup

Since this lab involves Java Database Connectivity (JDBC), additional setup is required beyond the standard PostgreSQL workflow.

3.1 System Installation

Open your terminal (`Ctrl+Alt+T`) and run the following commands to install the JDK and PostgreSQL (if not already installed):

```
# Update package list
sudo apt update

# Install default Java JDK
sudo apt install default-jdk

# Install PostgreSQL
sudo apt install postgresql postgresql-contrib

# Install VS Code (if not installed)
sudo snap install code --classic
```

3.2 VS Code Configuration

1. **Install Java Extensions:**
   - Open VS Code.
   - Click on the Extensions icon (box icon on the left sidebar).
   - Search for **"Extension Pack for Java"** by Microsoft and click Install.
2. **Get the PostgreSQL JDBC Driver:**
   - Download the driver JAR file from: `https://jdbc.postgresql.org/download/`
   - Save `postgresql-42.x.x.jar` to a known location (e.g., `Downloads`).
3. **Create Project & Add Driver:**
   - Press `Ctrl+Shift+P` → `Java: Create Java Project` → `No Build Tools`.
   - In the **Java Projects** view (bottom-left sidebar), find **Referenced Libraries**.
   - Click the + icon and select the `postgresql-42.x.x.jar` file you downloaded.

4 Understanding JDBC

4.1 What is JDBC?

JDBC stands for **Java Database Connectivity**. It is a standard Java API (Application Programming Interface) that allows Java applications to interact with relational databases. Before JDBC, writing database applications was difficult because different databases (like PostgreSQL, MySQL, Oracle) used different proprietary APIs. JDBC provides a uniform interface, meaning you can write one piece of Java code and, by simply changing the "Driver", connect it to any supported database.

4.2 How It Works: The Architecture

The JDBC architecture consists of three main layers working together:

1. **The Java Application (Your Code):** This is the code you write in VS Code. It uses standard Java classes like `Connection`, `Statement`, and `ResultSet`. Importantly, your code does not know the details of how PostgreSQL works internally; it only knows standard Java commands.
2. **The JDBC Driver (The Translator):** This is the `.jar` file you downloaded (e.g., `postgresql-42.x.x.jar`). It acts as a bridge or translator. When your Java code says "Execute Query", the Driver translates that generic request into the specific network protocol that the PostgreSQL server understands.
3. **The Database (PostgreSQL):** The actual storage engine running on your Ubuntu machine. It receives commands from the Driver, executes them, and returns data.

4.3 Core Components & Syntax

To connect to a database, you generally follow these 4 steps, which correspond to specific Java classes:

1. **Load the Driver:** `Class.forName("org.postgresql.Driver");`
2. **Establish Connection:** `DriverManager.getConnection(url, user, pass);`
3. **Create Statement:** `con.createStatement();`
4. **Execute & Process:** `stmt.executeQuery(sql);`

4.4 Prepared Statements

The `Statement` interface is sufficient for simple, static queries (like `SELECT * FROM table`). However, when your application needs to handle User Input (e.g., a user typing their ID to login), you must use a `PreparedStatement`.

Why use PreparedStatement?

1. **Security (SQL Injection Prevention):** If you simply concatenate strings (e.g., `"SELECT * FROM users WHERE name = " + input`), a malicious user can type code instead of a name to hack your database. `PreparedStatement` treats all user input strictly as data, never as executable code.

2. **Performance:** The database pre-compiles the SQL query structure once, making it faster to execute multiple times with different values.

4.5 Real-Life Application

In a real-world scenario, such as an ATM machine:

- When you insert your card (User Input), the Java application reads the card number.
- It uses JDBC to send a secure `SELECT pin FROM users WHERE card_id = ?` query to the bank's database using a PreparedStatement.
- The database returns the result, and the Java app compares it to the PIN you entered.

5 JDBC Code Examples

5.1 Basic Select (Using Statement)

This code demonstrates how to connect to the database and execute a simple, static query.

```
import java.sql.*;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://localhost:5432/cse4308_12345";
        String username = "student_12345";
        String password = "student12345";

        try {
            Class.forName("org.postgresql.Driver");
            Connection con = DriverManager.getConnection(url, username, password);
            System.out.println("Connected!");

            Statement statement = con.createStatement();
            ResultSet result = statement.executeQuery("SELECT * FROM account LIMIT 5")
    ;

            while (result.next()) {
                System.out.println("Account ID: " + result.getInt("a_id"));
            }

            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Listing 2: Reading Data (SELECT)

5.2 How to Run the Code

Method 1: VS Code (Recommended)
If you added the JAR file to "Referenced Libraries", click the **Run** button above `main`.

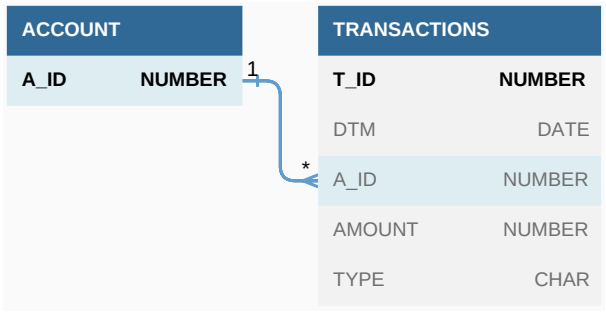
Method 2: Terminal Command (Manual)

```
# Compile
javac -cp .:postgresql-42.7.3.jar Main.java
# Run
java -cp .:postgresql-42.7.3.jar Main
```

6 Scenario

Consider the (partial) schema of a Banking Management System.

- **ACCOUNT** table: Contains only the account ID.
- **TRANSACTIONS** table: Stores transaction ID, date, account involved, amount, and type.
- **TYPE** column: 0 for credit (add money), 1 for debit (subtract money).



There are 3 types of accounts in the bank:

1. **Commercially Important Person (CIP):** Total transaction volume $> 5,000,000$.
2. **Very Important Person (VIP):** Total transaction volume between $2,500,000 - 4,500,000$.
3. **Ordinary Person (OP):** Total transaction volume $< 1,000,000$.

7 Lab Task

First Load the new sql file for the task. Then write a JAVA code to:

1. Count the total number of transactions conducted under account 49.
2. Count the number of credit transactions.
3. List the transactions that occurred in the last 6 months of 2021.
4. Count the number of CIP, VIP, and OPs. Also show the number of people that do not fall in any of the categories.
5. **Safe Insert Task:** Create a Java class that prompts the user for inputs (Account ID, Amount, Type) and inserts a new transaction using a `PreparedStatement`. Use the current date for the transaction.

8 Useful Syntax & Coding Guidelines

Use the following snippets to help you implement the tasks.

8.1 Retrieving Values from ResultSet

When iterating through your query results in Java, ensure you match the Java variable type with the database column type.

```
while (result.next()) {
    // Get integer value (e.g., account_id, type)
    int accId = result.getInt("account_id");

    // Get decimal/money value
    double amount = result.getDouble("amount");

    // Get date value
    Date date = result.getDate("txn_date");
}
```

8.2 Filtering by Date

For Task 3 (last 6 months of 2021), you can filter directly in your SQL string using the `WHERE` clause and standard date formats (`'YYYY-MM-DD'`).

```
String sqlQuery = "SELECT * FROM transactions " +
                  "WHERE txn_date >= '2021-07-01' " +
                  "AND txn_date <= '2021-12-31'";
```

8.3 Calculating Balance & Volume

Since the `ACCOUNT` table does not store the balance, you must calculate it by fetching all transactions for a specific user and processing them in Java.

- **Balance:** Start at 0. Add `amount` if `type == 0` (Credit). Subtract `amount` if `type == 1` (Debit).
- **Volume:** Sum of all `amount`s, regardless of type.

```
double balance = 0;
double totalVolume = 0;

if (type == 0) { // Credit
    balance += amount;
} else if (type == 1) { // Debit
    balance -= amount;
}
totalVolume += amount;
```

8.4 PreparedStatement Syntax

To solve the safe insert task, you must use ? placeholders instead of concatenating strings.

```
// 1. Write SQL with placeholders
String insertSQL = "INSERT INTO transactions (txn_date, a_id, amount, type) VALUES (?,
    ?, ?, ?)";

// 2. Create PreparedStatement
PreparedStatement pstmt = con.prepareStatement(insertSQL);

// 3. Fill the placeholders (Index starts at 1)
// Example: Setting the date to today
long millis = System.currentTimeMillis();
pstmt.setDate(1, new java.sql.Date(millis));

// Example: Setting the Account ID (e.g., from user input)
pstmt.setInt(2, 49);

// Example: Setting the Amount and Type
pstmt.setDouble(3, 500.00);
pstmt.setInt(4, 0);

// 4. Execute the Insert
pstmt.executeUpdate();
```

9 PostgreSQL in Python

While this lab focuses on Java (JDBC), real-world applications often use Python for database interaction, especially in Data Science and Web Development (e.g., Django, Flask).

In Python, the most popular driver is `psycopg2`. The logic remains very similar to JDBC: you need a Connection, a Cursor (similar to Statement), and you Execute queries.

9.1 1. Installation

Unlike Java where you add a JAR file, in Python you install the package via terminal:

```
pip install psycopg2-binary
```

9.2 2. Python Code Example

This script connects to the same `cse4308_12345` database (as user `student_12345`) you created earlier.

```
import psycopg2

try:
    # 1. Establish Connection
    connection = psycopg2.connect(
        user="student_12345",
        password="student12345",
        host="localhost",
        port="5432",
        database="cse4308_12345"
    )

    # 2. Create Cursor (Similar to JDBC Statement)
    cursor = connection.cursor()

    # 3. Execute Query
    cursor.execute("SELECT * FROM account LIMIT 5;")

    # 4. Fetch Results (Similar to ResultSet)
    records = cursor.fetchall()

    print("Printing first 5 accounts:")
    for row in records:
        print("Account ID:", row[0])

except (Exception, psycopg2.Error) as error:
    print("Error while connecting to PostgreSQL", error)

finally:
    # 5. Close Connection
    if connection:
        cursor.close()
        connection.close()
        print("PostgreSQL connection is closed")
```